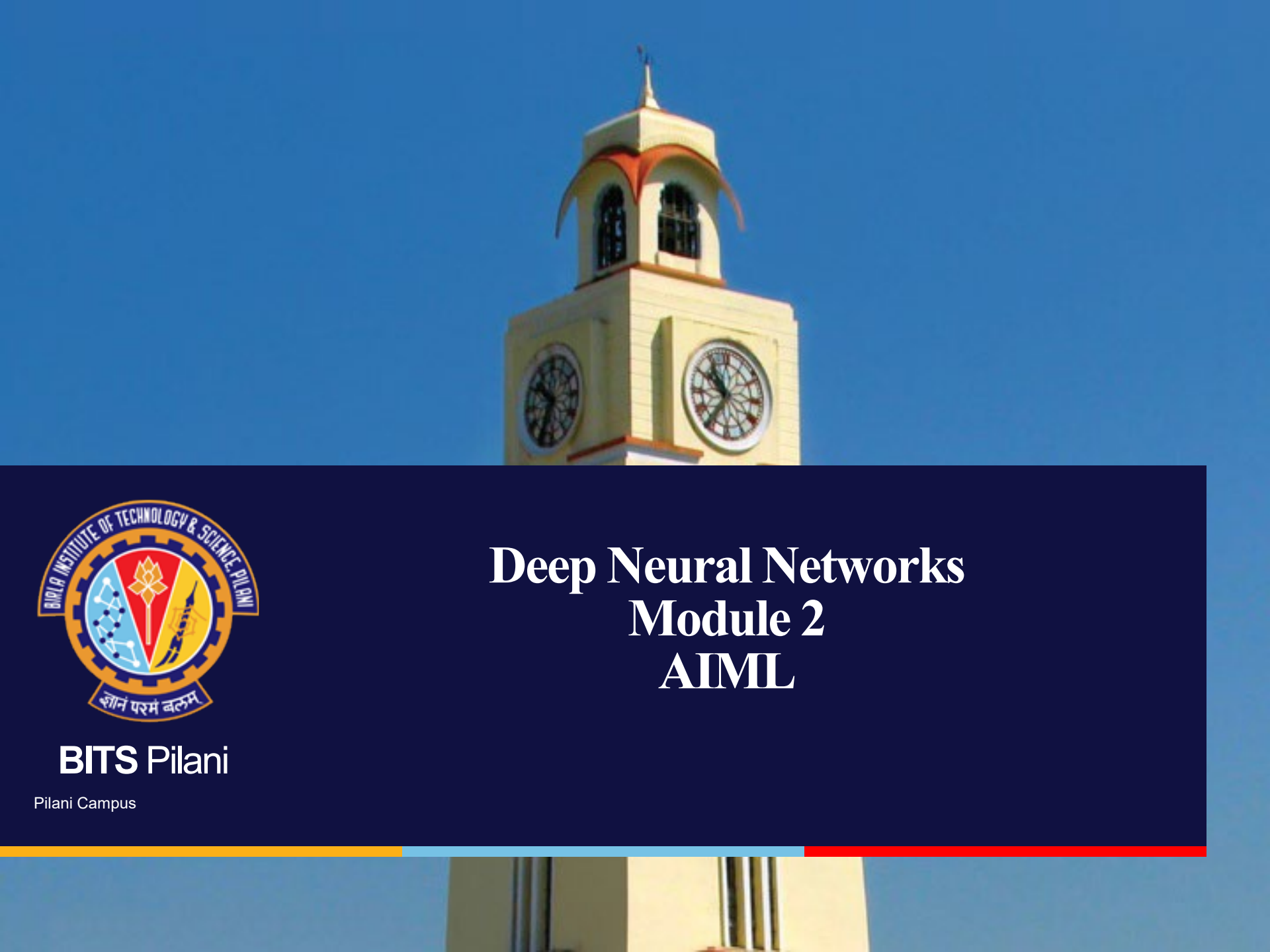


Deep Neural Networks
Module 2
AIML

BITS Pilani

Pilani Campus

Deep Neural Network

innovate achieve lead

Disclaimer and Acknowledgement

- The content for these slides has been obtained from books and various other source on the Internet
- I here by acknowledge all the contributors for their material and inputs.
- I have provided source information wherever necessary
- I have added and modified the content to suit the requirements of the course

Session Agenda

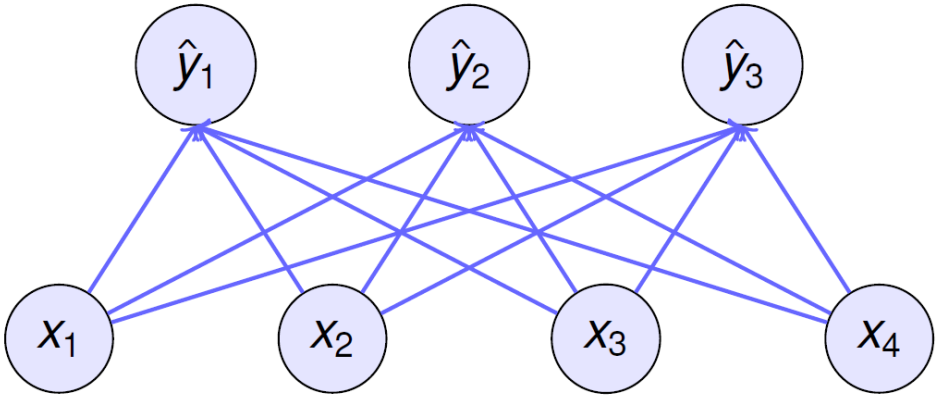
- Multi-class Classification
- Deep Feedforward Neural Networks
- Activation Functions
- Computational Graph
- Forward Propagation

Multi-class Classification Example

- Each input consists of a 2×2 grayscale image.
- Represent each pixel value with a single scalar, giving four features x_1, x_2, x_3, x_4.
- Assume that each image belongs to one among the categories ”square”, ”triangle”, and ”circle”.
- How to represent the labels?
  - Use label encoding.
    $y \in 1, 2, 3$, where the integers represent *circle*, *square*, *triangle* respectively.
  - Use one-hot encoding.
    $y \in (1, 0, 0), (0, 1, 0), (0, 0, 1)$. y would be a three-dimensional vector, with (1, 0, 0) corresponding to ”circle’, (0, 1, 0) to ”square”, and (0, 0, 1) to ”triangle”.

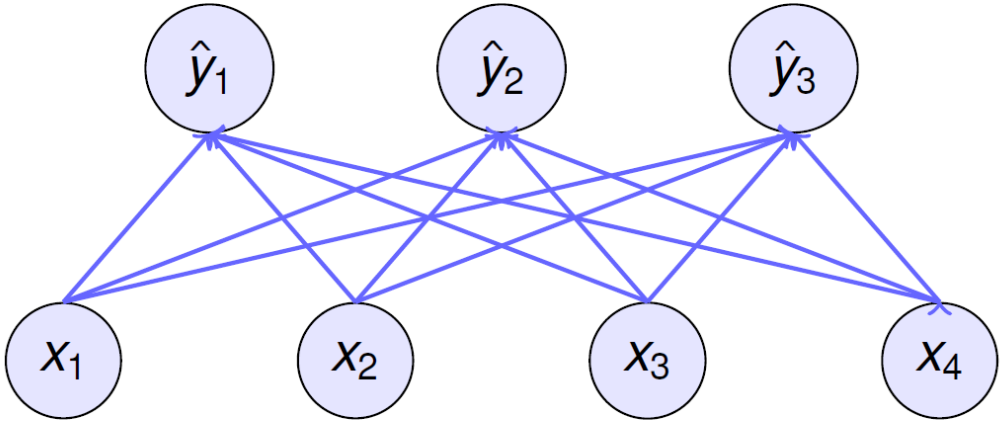
Single-Layer Neural Network

- A model with multiple outputs, one per class. Each output will correspond to its own affine function.
- 4 features and 3 possible output categories
- Every input is connected to every output, This transformation is a **fully-connected layer or dense layer.**



Single-Layer Neural Network

- 12 scalars to represent the weights and 3 scalars to represent the biases .
- Compute three logits, $\hat{y}_1$, $\hat{y}_2$, and $\hat{y}_3$, for each input.
- Weights is a 3×4 matrix and bias is 1×3 matrix.



$$z_1 = x_1 w_{11} + x_2 w_{12} + x_3 w_{13} + x_4 w_{14} + b_1 \qquad \hat{y}_1 = softmax(z_1)$$
$$z_1 = x_1 w_{21} + x_2 w_{22} + x_3 w_{23} + x_4 w_{24} + b_2 \qquad \hat{y}_2 = softmax(z_2)$$
$$z_3 = x_1 w_{31} + x_2 w_{32} + x_3 w_{33} + x_4 w_{34} + b_3 \qquad \hat{y}_3 = softmax(z_3)$$

Softmax Operation

Example

Suppose the raw scores (logits) are $z = [2.0, 1.0, 0.1]$.

1. Calculate the exponential values:

$$e^z = [e^{2.0}, e^{1.0}, e^{0.1}] \approx [7.39, 2.72, 1.11]$$

2. Sum the exponential values:

$$\sum e^z = 7.39 + 2.72 + 1.11 \approx 11.22$$

3. Normalize each value:

$$\text{softmax}(z) = \left[\frac{7.39}{11.22}, \frac{2.72}{11.22}, \frac{1.11}{11.22}\right] \approx [0.658, 0.242, 0.099]$$

Thus, the probabilities are $[0.658, 0.242, 0.099]$, indicating that the first class is the most likely.

- Interpret the outputs of our model as probabilities.
- Any output $\hat{y}_j$ is interpreted as the probability that a given item belongs to class j. Then choose the class with the largest output value as our prediction $argmax_j \hat{y}_j$.
- If $\hat{y}_1$, $\hat{y}_2$, and $\hat{y}_3$ are 0.1, 0.8, and 0.1, respectively, then predict category 2.
- The softmax function transforms the outputs such that they become non-negative and sum to 1, while requiring that the model remains differentiable.

$$\hat{y} = softmax(Z) \quad where \quad \hat{y}_j = \frac{exp(z_j)}{\sum_k exp(z_k)}$$

- First exponentiate each logit (ensuring non-negativity) and then divide by their sum (ensuring that they sum to 1).
- Softmax is a non-linear function.

Log-Likelihood Loss / Cross-Entropy Loss

- The softmax function gives a vector $\hat{y}$, which can be interpreted as estimated conditional probabilities of each class given any input x,

$$\hat{y}_j = P(y = class_j \mid x)$$

- Compare the estimates with reality by checking how probable the actual classes are according to our model, given the features:

$$P(Y \mid X) = \prod_{i=1}^{m} P(y^{(i)} \mid x^{(i)})$$

Log-Likelihood Loss / Cross-Entropy Loss

- Maximize $P(Y \mid X)$ = Minimize the negative log-likelihood

$$-\log P(Y \mid X) = \sum_{i=1}^{m} -\log P(y^{(i)} \mid x^{(i)}) = \sum_{i=1}^{m} loss\, P(y^{(i)}, \hat{y}^{(i)})$$

$$loss\, P(y^{(i)}, \hat{y}^{(i)}) = -\sum_{i=1}^{m} y_j \log \hat{y}_j$$

1. Recap of the Softmax Output

The logits were $z = [2.0, 1.0, 0.1]$, and the softmax probabilities (computed earlier) are:

$$\text{softmax}(z) \approx [0.658, 0.242, 0.099]$$

2. Log-Likelihood Loss Formula

For a single data point, the **log-likelihood loss** is:

$$\text{Loss} = -\log(\hat{p}_{\text{true}})$$

where:

- $\hat{p}_{\text{true}}$ is the predicted probability for the correct class (true label).

3. Example with a True Class

Assume the true class is $y = 0$ (the first class in our case).

- The predicted probability for this class (from the softmax output) is $\hat{p}_{\text{true}} = 0.658$.

4. Compute Log-Likelihood Loss

Take the negative logarithm of the probability for the true class:

$$\text{Loss} = -\log(0.658)$$

Using a calculator or logarithm:

$$\log(0.658) \approx -0.419$$

So the loss becomes:

$$\text{Loss} = -(-0.419) = 0.419$$

Softmax Example

- z_1 = Un-normalized log probabilities
- e^{z_1} = Un-normalized probabilities
- $\hat{y}_i = softmax(z_1)$ = normalized probabilities
- L_i = loss = $y_i \log \hat{y}_i$

class	z_1	e^{z_1}	$softmax(z_1)$	L_1
cat	3.2	24.5	0.13	0.89
car	5.1	164.0	0.87	0.06
dog	-1.7	0.18	0.00	∞

Deep Feedforward Neural Networks

- A neural network with more number of hidden layers is considered as a **deep neural network**. There are no feedback connections.
- Convolutional networks are examples of feedforward neural networks.

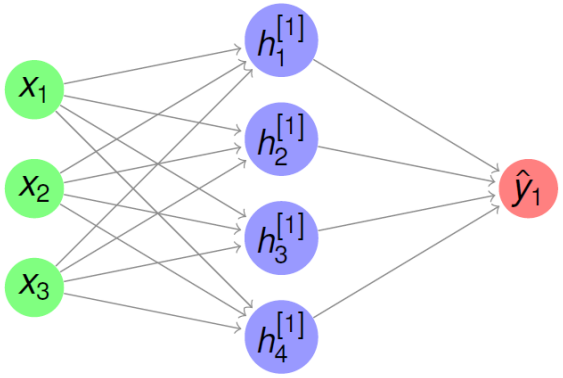


FIGURE: Shallow Neural Network

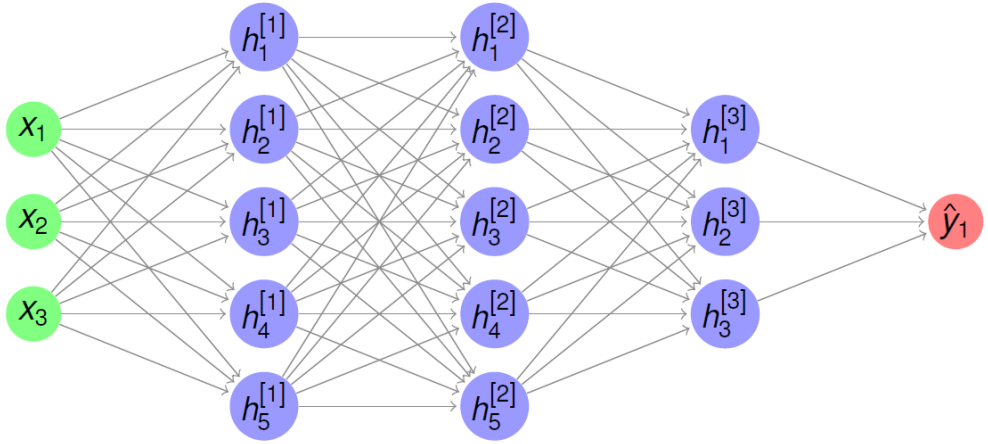
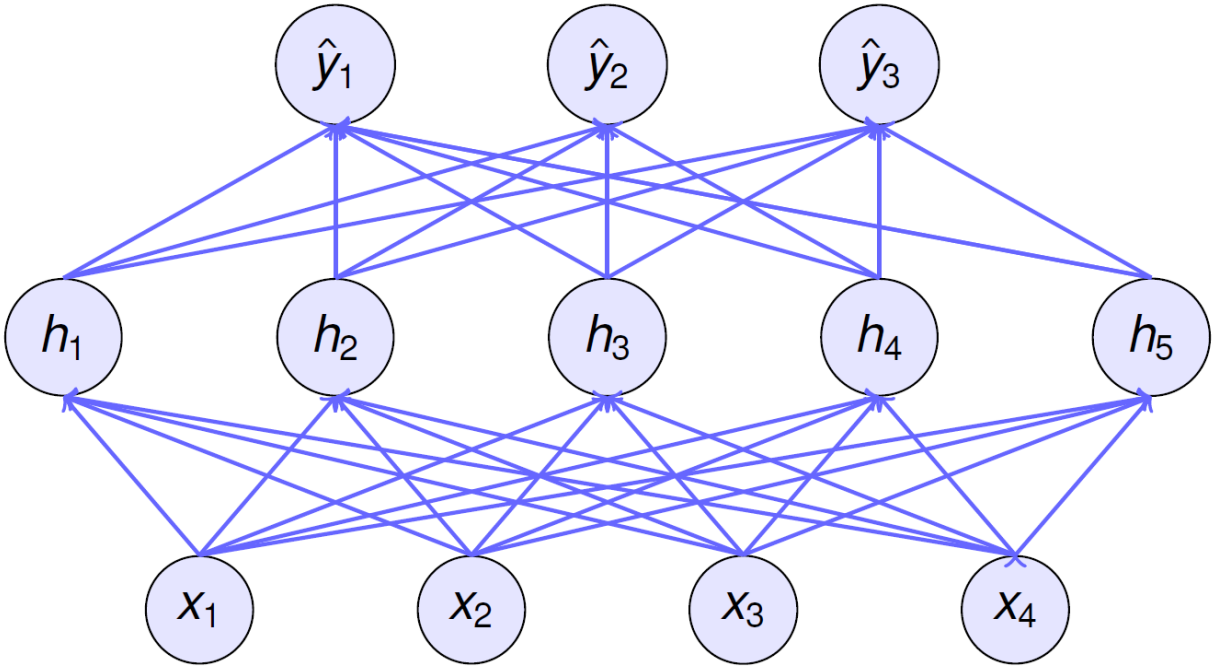


FIGURE: Deep Neural Network

Deep Feedforward Neural Networks

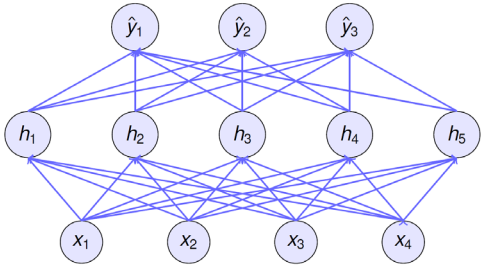
- With deep neural networks, use the data to jointly learn both a representation via hidden layers and a linear predictor that acts upon that representation.
- Add many hidden layers by stacking many fully-connected layers on top of each other. Each layer feeds into the layer above it, until we generate outputs.
- The first $(L-1)$ layers learns the representation and the final layer is the linear predictor.

DNN Architecture



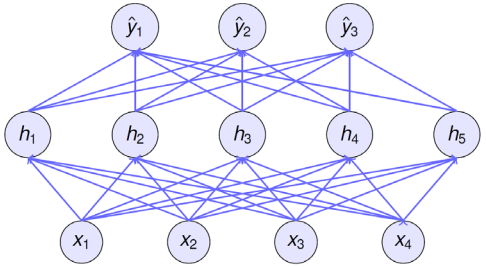
- DNN has 4 inputs, 3 outputs, and its hidden layer contains 5 hidden units.
- Number of layers in this DNN is 2.

DNN Architecture



- The layers are fully connected.
- Every input influences every neuron in the hidden layer.
- Each of the hidden neurons in turn influences every neuron in the output layer.
- The outputs of the hidden layer are called as **hidden representations** or hidden-layer variable or a hidden variable.

DNN Architecture



HIDDEN LAYERS – intermediate layers, desired output for each of these layers are not shown.

DEPTH – number of layers.

WIDTH – dimensionality of the hidden layers

DNN - General Strategy

- Design a DNN architecture of the network. Architecture depends on the problem / application / complexity of the decision boundary.
- Choose the activation functions that will be used to compute the hidden layer activations. The activation function is the same for all neurons in a layer. Activation function can change between layers.
- Choose the cost function. It depends on whether regression, binary classification or multi-class classification.
- Choose the optimizer algorithm, depends on the complexity and variance of the input data.
- Train the feedforward network. Learning in deep neural networks requires computing the gradients using the back-propagation algorithm.
- Evaluate the performance of the network.

The Flow of Computations in Neural Networks

- The flow of computations in a neural network goes in two ways:
  - Left-to-right: This is referred to as *forward propagation*, which results in computing the output of the network
  - Right-to-left: This is referred to as *back propagation*, which results in computing the gradients (or derivatives) of the parameters in the network

- The intuition behind this 2-way flow of computations can be explained through the concept of "computation graphs"

Computation Graphs

- Each node in the graph indicate a variable.
- An operation is a simple function of one or more variable.
- An operation is defined such that it returns only a single output variable.
- If a variable y is computed by applying an operation to a variable x , then we draw a directed edge from x to y .

Computation Graphs Example

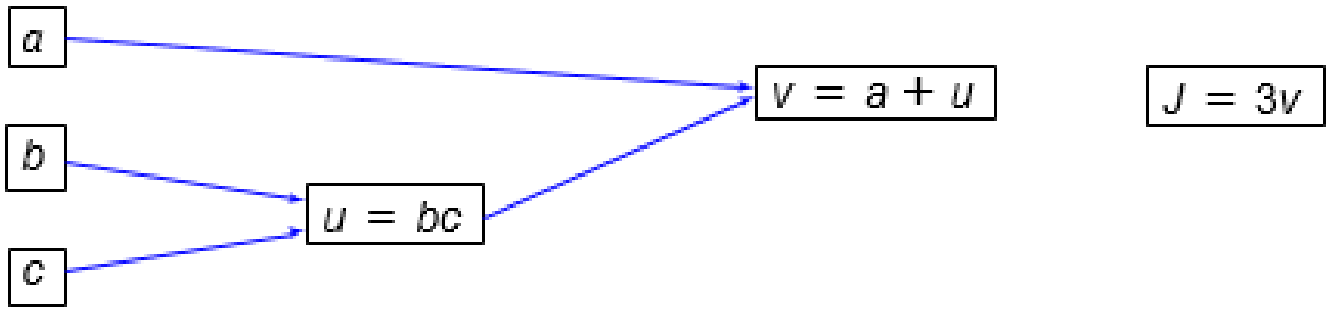
Forward Pass:

$$\text{Let } J(a, b, c) = 3(a + bc)$$

$$u = bc$$

$$v = a + u$$

$$\text{Then } J = 3v$$



Forward propagation allows computing ***J***

Computation Graphs Example

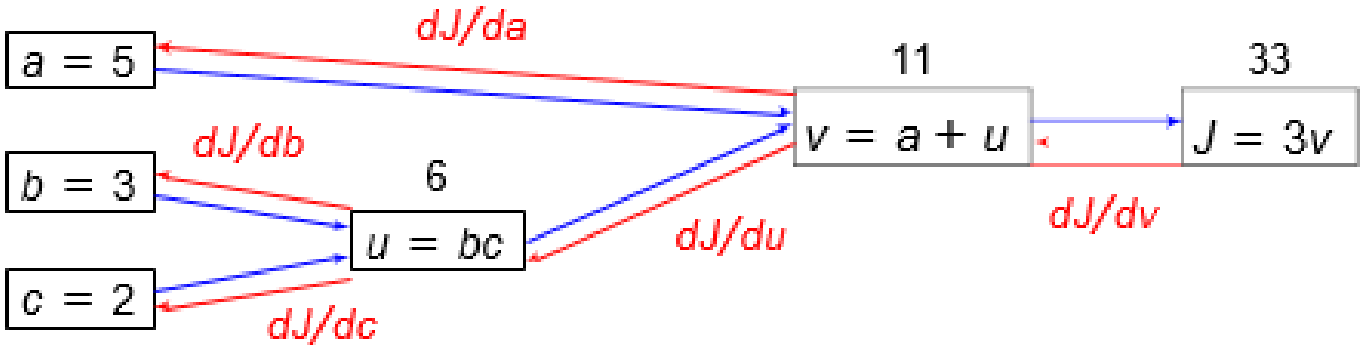
Backward Pass / Back propagation of gradients:

Let $J(a, b, c) = 3(a + bc)$

$u = bc$

$v = a + u$

Then $J = 3v$



To compute the derivative of J with respect to v, we went *back* to v, nudged it, and measured the corresponding resultant increase on J

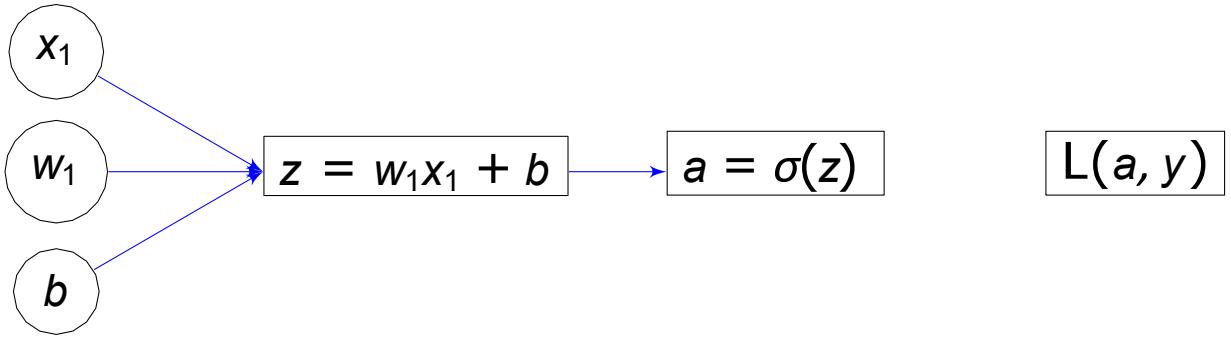
Computation Graph – Binary Classification

Forward Pass:

$$z = w^{\mathrm{T}} X + b$$

$$a = y = \sigma(z)$$

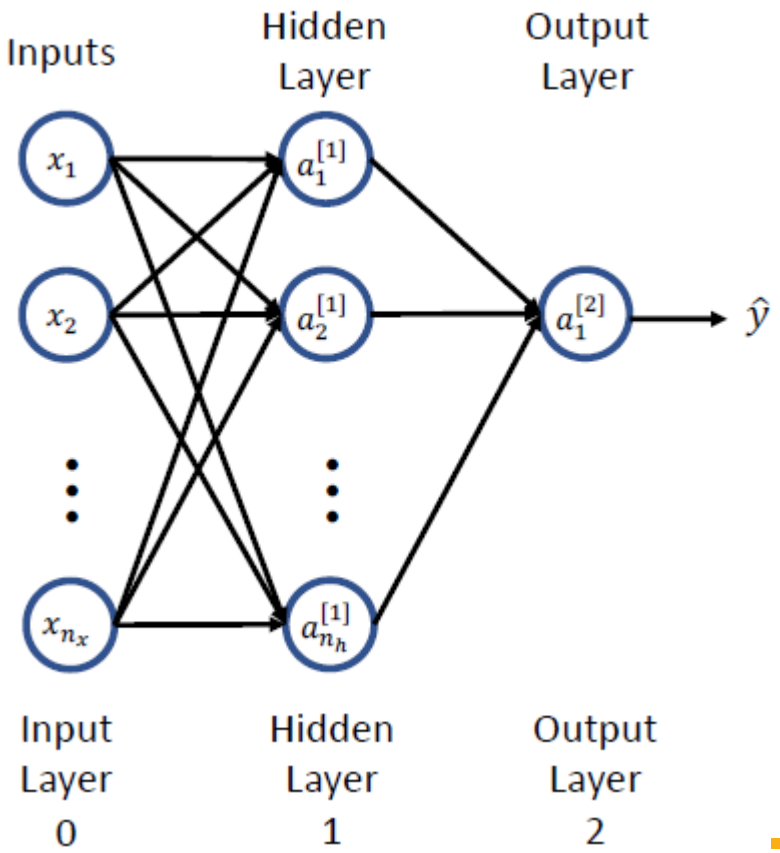
$$\mathrm{L}(a, y) = -[y \log a + (1 - y) \log(1 - a)]$$



Formalizing Neural Network Construction

$$Y = f_N(\mathbf{W}_N f_{N-1}(\dots f_2(\mathbf{W}_2 f_1(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2)\dots) + \mathbf{b}_N)$$

Let's consider a 2-layer Network



Output of one activation unit

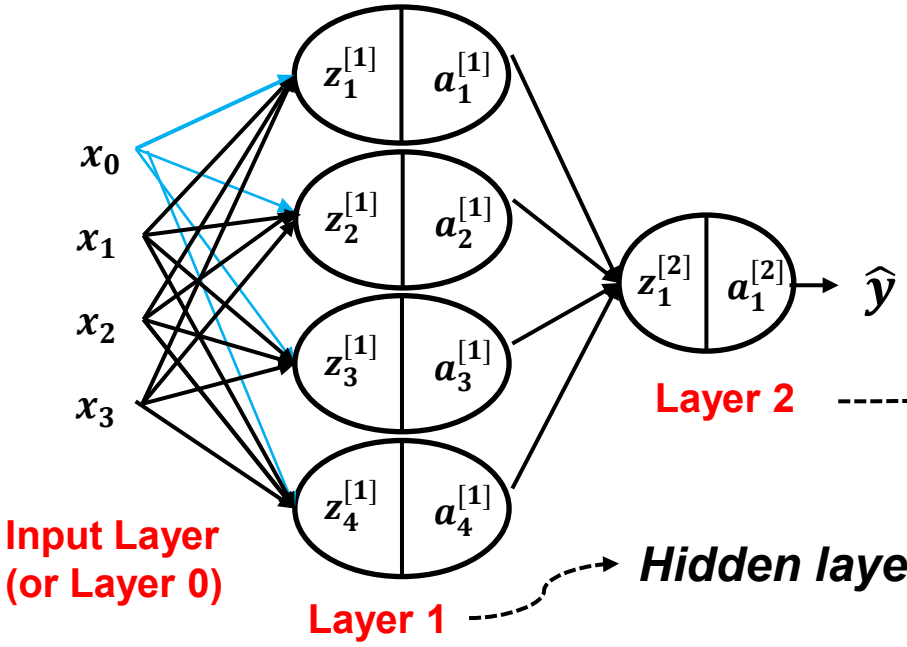
$$z_i^{[l]} = \sum_{j=1}^{n_h^{[l-1]}} w_{i,j}^{[l]} a_j^{[l-1]} + b_i^{[l]}$$

$$a_i^{[l]} = g\left(z_i^{[l]}\right)$$

Where

- $g()$ = activation function
- $w^{[l]}$ = weight parameter of input $a^{[l-1]}$ going to unit $a^{[l]}$
- n_x = number of features (model inputs)
- n_h = number neurons/units in hidden layer
- $x_i = a_i^{[0]}$

2-layer NN



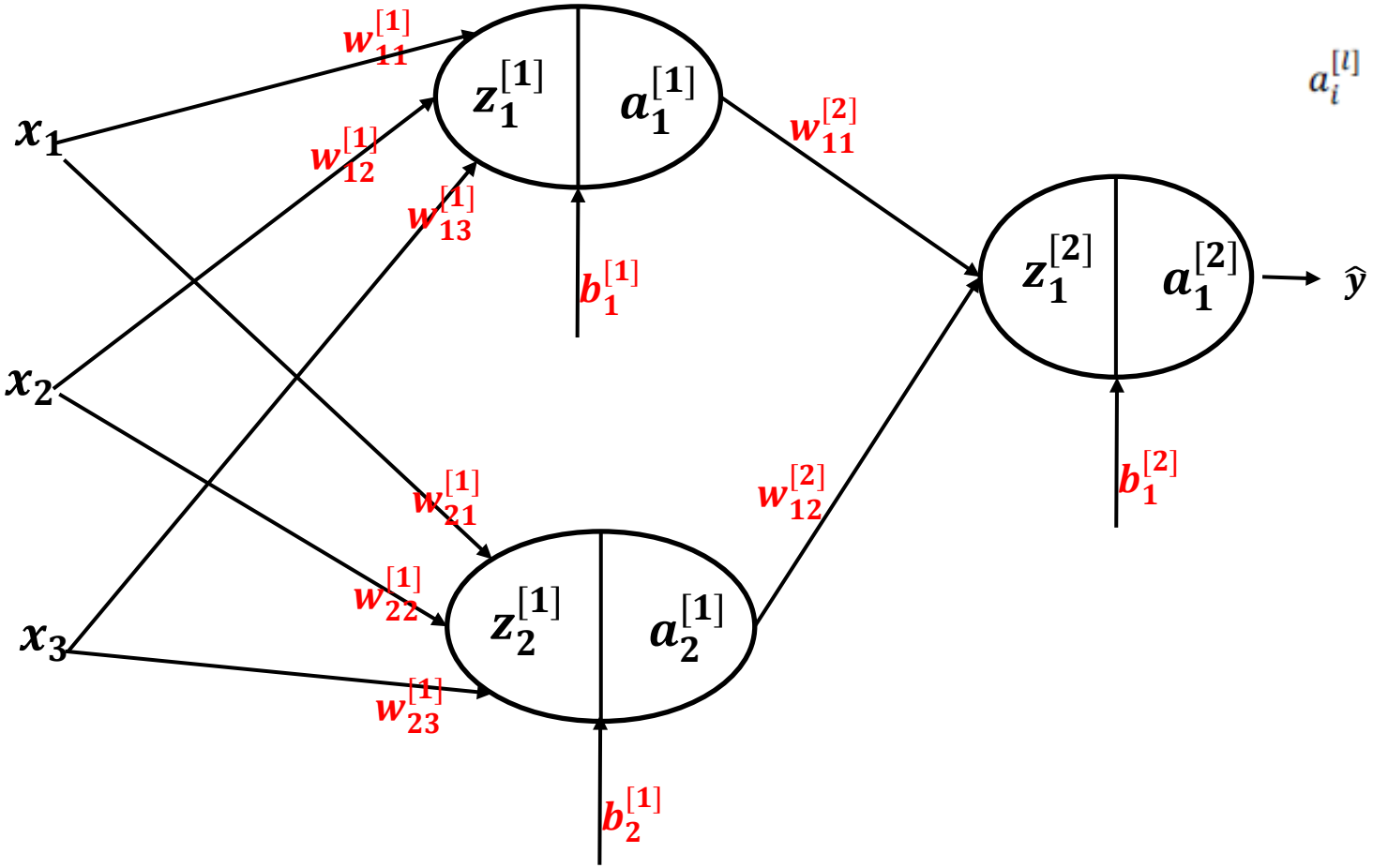
Interestingly, neural networks *learn* their own features!

Example (without vectorization)

innovate achieve lead

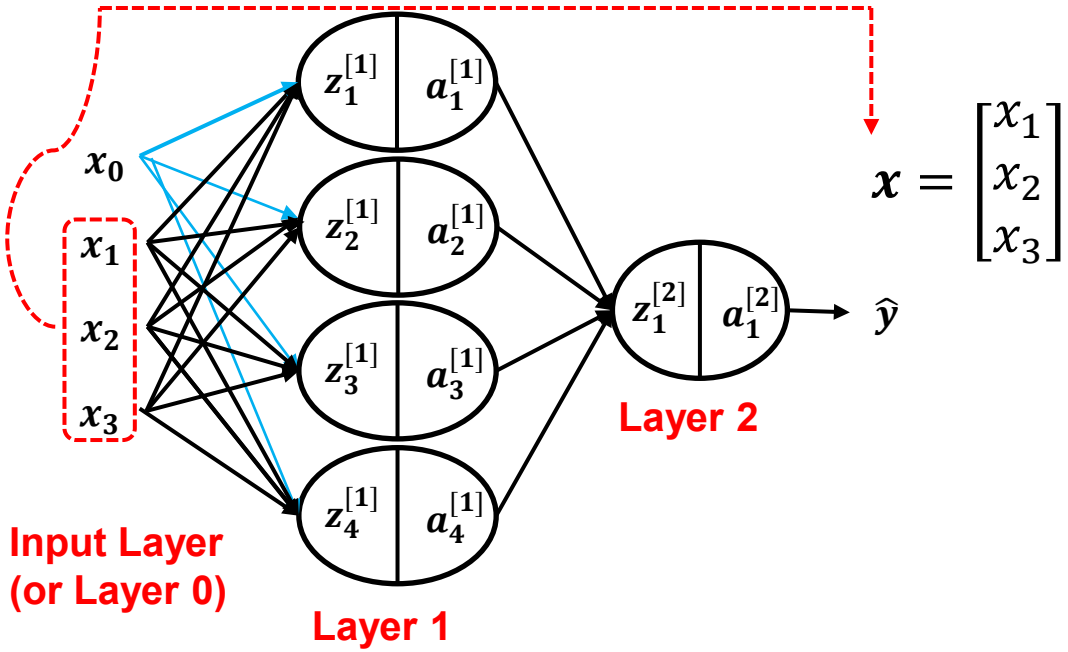
$$z_i^{[l]} = \sum_{j=1}^{n_h^{[l-1]}} w_{i,j}^{[l]} a_j^{[l-1]} + b_i^{[l]}$$

$$a_i^{[l]} = g\left(z_i^{[l]}\right)$$

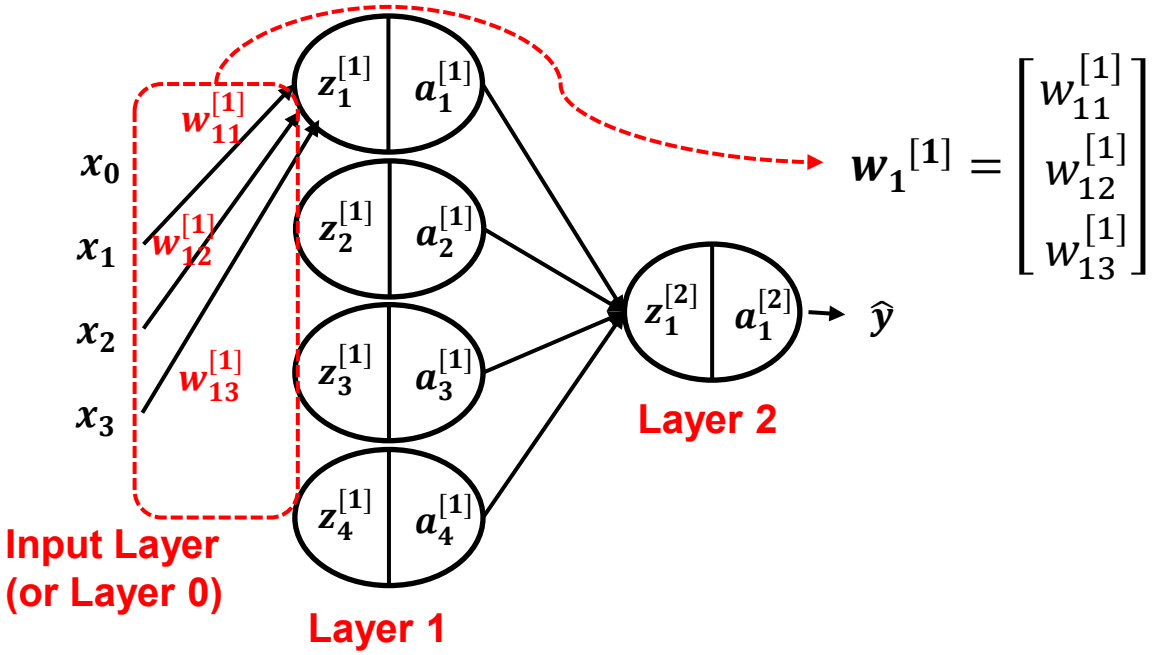


Vectorizing Input and All Variables

To help develop an efficient learning algorithm, let us *vectorize* (represent in vectors & matrices) the input and the variables involved



Vectorizing Input and All Variables

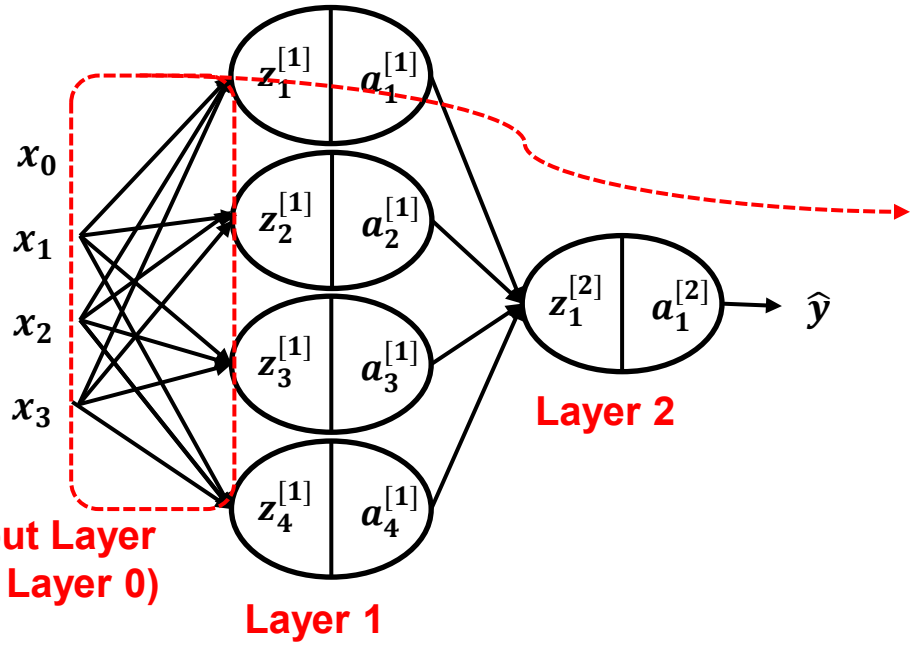


Vectorizing Input and All Variables

$$w_1^{[1]} = \begin{bmatrix} w_{11}^{[1]} \\ w_{12}^{[1]} \\ w_{13}^{[1]} \end{bmatrix}$$

$$w^{[1]} = \begin{bmatrix} \cdots & w_1^{[1]T} & \cdots \\ \cdots & w_2^{[1]T} & \cdots \\ \cdots & w_3^{[1]T} & \cdots \\ \cdots & w_4^{[1]T} & \cdots \end{bmatrix}$$

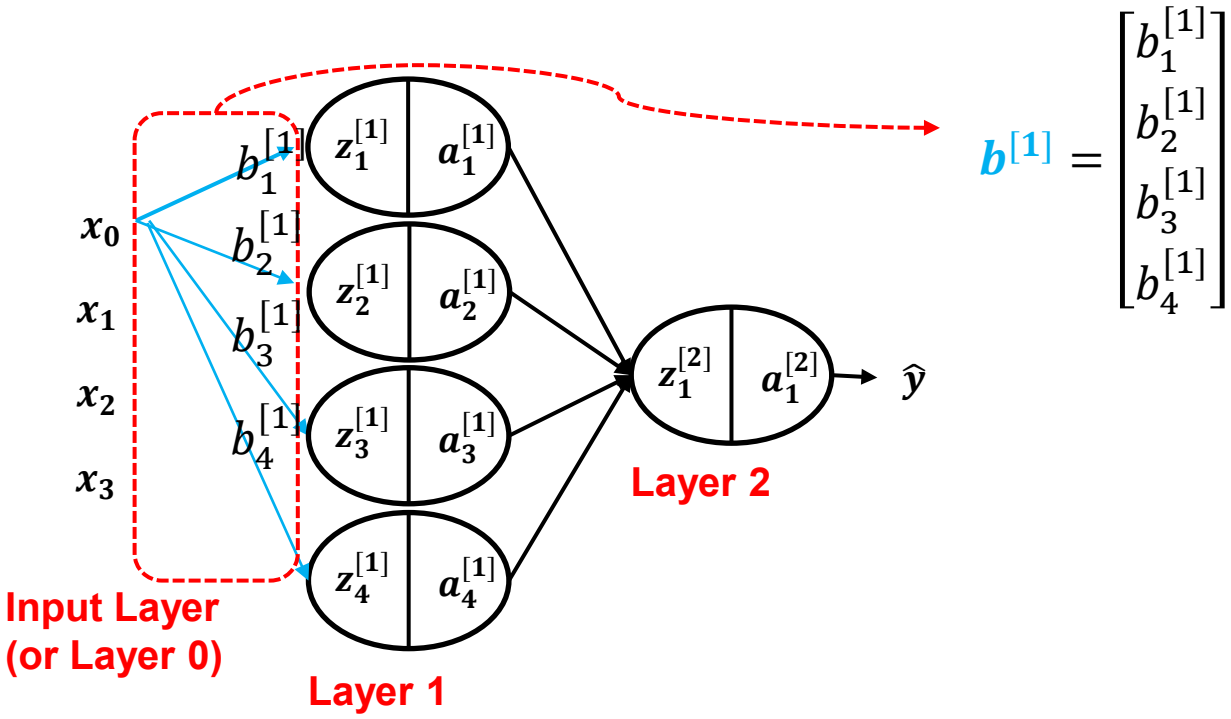
4 × 3



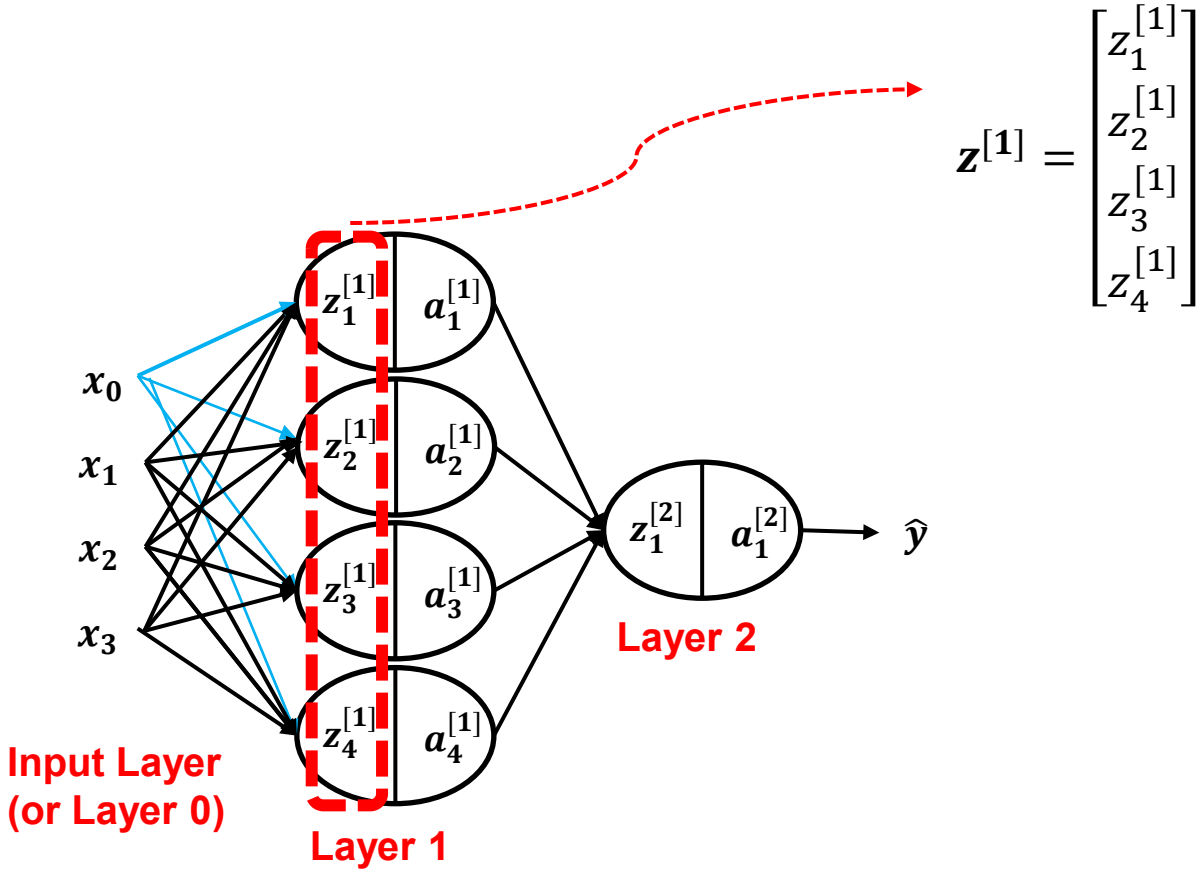
$$w^{[1]} = \begin{bmatrix} w_{11}^{[1]} & w_{12}^{[1]} & w_{13}^{[1]} \\ w_{21}^{[1]} & w_{22}^{[1]} & w_{23}^{[1]} \\ w_{31}^{[1]} & w_{32}^{[1]} & w_{33}^{[1]} \\ w_{41}^{[1]} & w_{42}^{[1]} & w_{43}^{[1]} \end{bmatrix}$$

Dimension of $w^{[1]}$ = (4, 3)

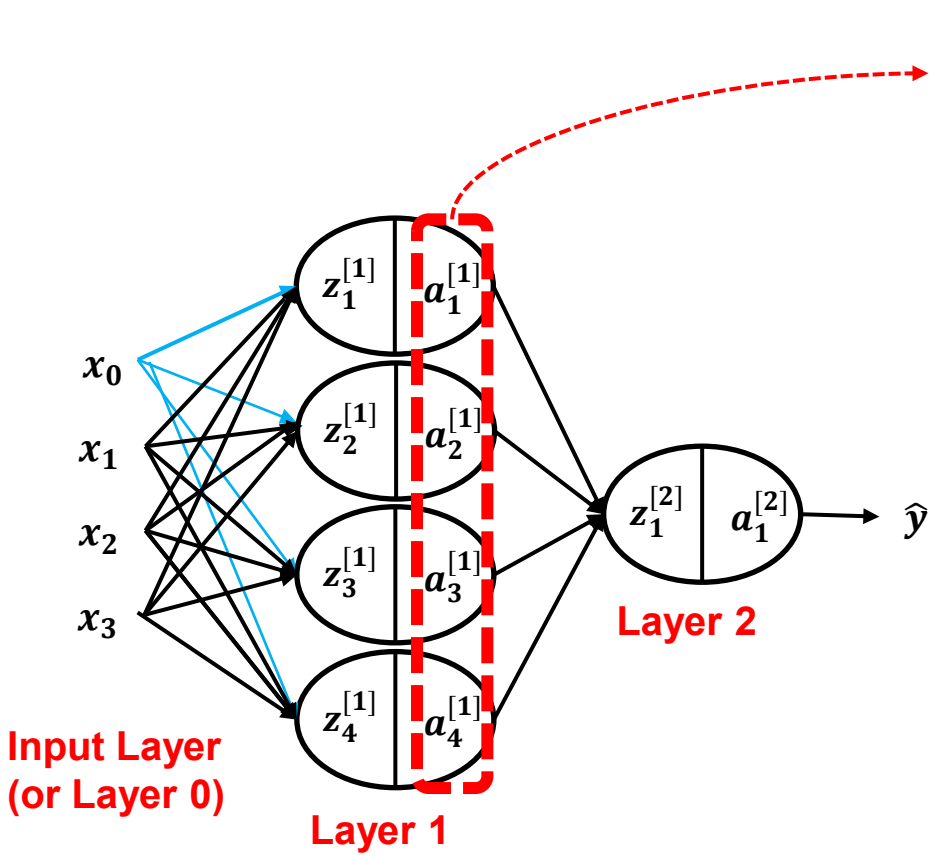
Vectorizing Input and All Variables



Vectorizing Input and All Variables

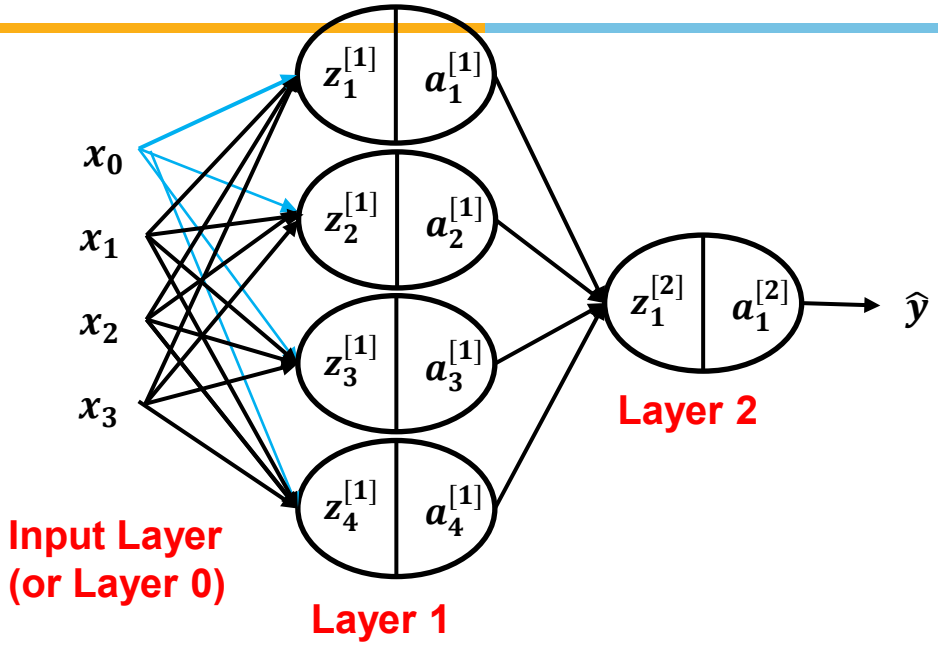


Vectorizing Input and All Variables



Vectorizing Input and All Variables

$$z^{[1]} = w^{[1]}x + b^{[1]}$$



$$z^{[1]} = \begin{bmatrix} w_{11}^{[1]} & w_{12}^{[1]} & w_{13}^{[1]} \\ w_{21}^{[1]} & w_{22}^{[1]} & w_{23}^{[1]} \\ w_{31}^{[1]} & w_{32}^{[1]} & w_{33}^{[1]} \\ w_{41}^{[1]} & w_{42}^{[1]} & w_{43}^{[1]} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \\ b_4^{[1]} \end{bmatrix}$$

$$z^{[1]} = \begin{bmatrix} w_{11}^{[1]}x_1 + w_{12}^{[1]}x_2 + w_{13}^{[1]}x_3 \\ w_{21}^{[1]}x_1 + w_{22}^{[1]}x_2 + w_{23}^{[1]}x_3 \\ w_{31}^{[1]}x_1 + w_{32}^{[1]}x_2 + w_{33}^{[1]}x_3 \\ w_{41}^{[1]}x_1 + w_{42}^{[1]}x_2 + w_{43}^{[1]}x_3 \end{bmatrix} + \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \\ b_4^{[1]} \end{bmatrix}$$

In matrix form

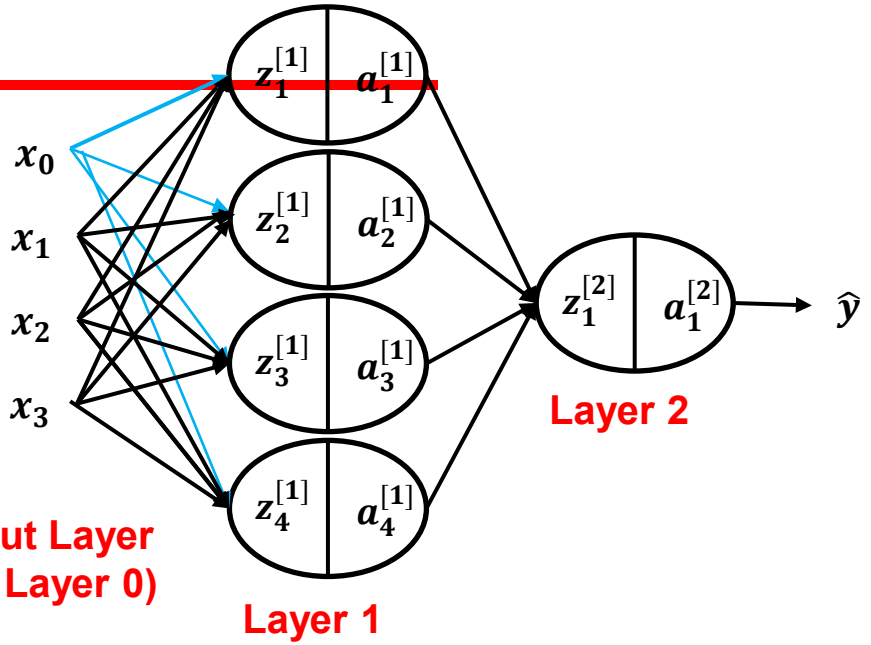
$$\begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \\ z_4^{[1]} \end{bmatrix} = \underbrace{\begin{bmatrix} \cdots w_1^{[1]T} \cdots \\ \cdots w_2^{[1]T} \cdots \\ \cdots w_3^{[1]T} \cdots \\ \cdots w_4^{[1]T} \cdots \end{bmatrix}}_{4 \times 3} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}}_{3 \times 1} + \underbrace{\begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \\ b_4^{[1]} \end{bmatrix}}_{4 \times 1}$$

$$z^{[1]} = \begin{bmatrix} w_{11}^{[1]}x_1 + w_{12}^{[1]}x_2 + w_{13}^{[1]}x_3 + b_1^{[1]} \\ w_{21}^{[1]}x_1 + w_{22}^{[1]}x_2 + w_{23}^{[1]}x_3 + b_2^{[1]} \\ w_{31}^{[1]}x_1 + w_{32}^{[1]}x_2 + w_{33}^{[1]}x_3 + b_3^{[1]} \\ w_{41}^{[1]}x_1 + w_{42}^{[1]}x_2 + w_{43}^{[1]}x_3 + b_4^{[1]} \end{bmatrix}$$

Vectorizing Input and All Variables

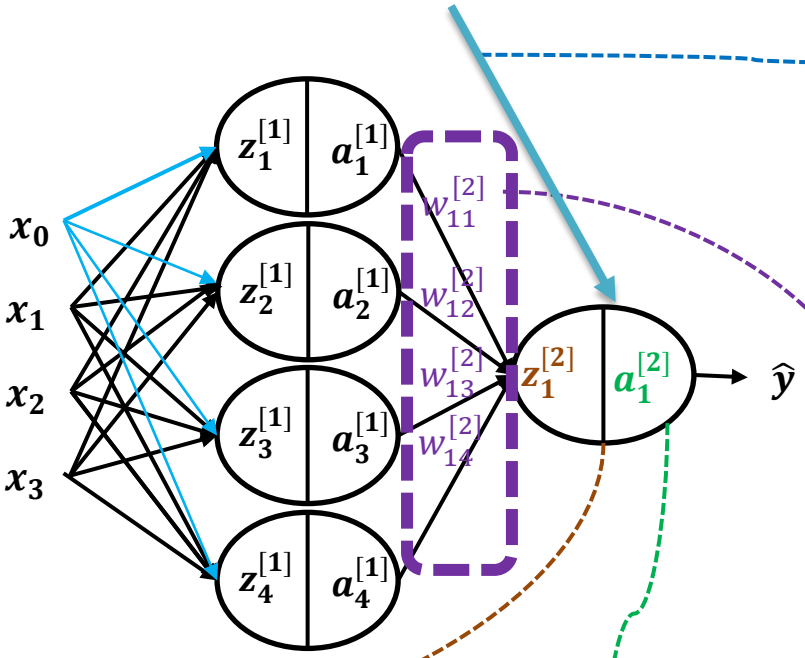
$$a^{[1]} = \sigma(z^{[1]})$$

$$z^{[1]} = w^{[1]}x + b^{[1]}$$



$$z^{[1]} = \begin{bmatrix} w_{11}^{[1]}x_1 + w_{12}^{[1]}x_2 + w_{13}^{[1]}x_3 + b_1^{[1]} \\ w_{21}^{[1]}x_1 + w_{22}^{[1]}x_2 + w_{23}^{[1]}x_3 + b_2^{[1]} \\ w_{31}^{[1]}x_1 + w_{32}^{[1]}x_2 + w_{33}^{[1]}x_3 + b_3^{[1]} \\ w_{41}^{[1]}x_1 + w_{42}^{[1]}x_2 + w_{43}^{[1]}x_3 + b_4^{[1]} \end{bmatrix} \dashrightarrow (z_1^{[1]}) \longrightarrow a_1^{[1]} = \sigma(z_1^{[1]})$$

Vectorizing Input and All Variables



$$b^{[2]} = \left[b_1^{[2]}\right]$$

$$w_1^{[2]} = \begin{bmatrix} w_{11}^{[2]} \\ w_{12}^{[2]} \\ w_{13}^{[2]} \\ w_{14}^{[2]} \end{bmatrix}$$

$$w^{[2]} = \left[\ldots\ldots w_1^{[2]} \ldots\ldots\right]$$

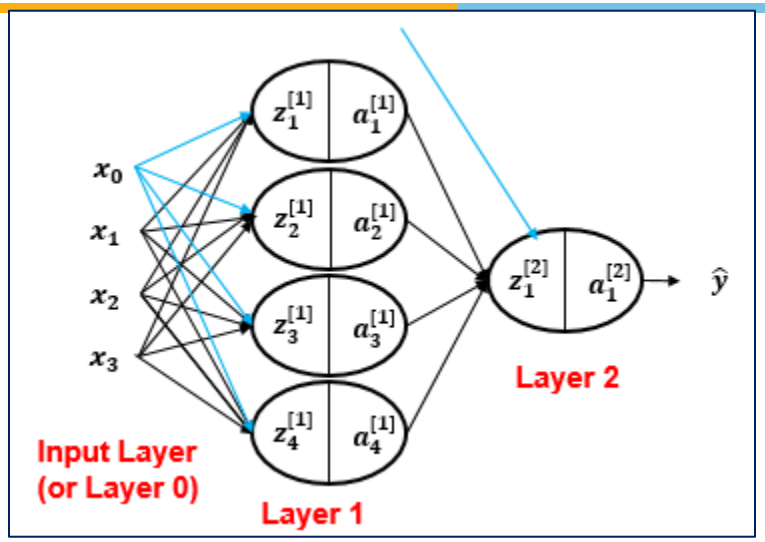
$$w^{[2]} = \left[w_{11}^{[2]}\ w_{12}^{[2]}\ w_{13}^{[2]}\ \ w_{14}^{[2]}\right]$$

$$z^{[2]} = \left[z_1^{[2]}\right]$$

$$a^{[2]} = \left[a_1^{[2]}\right]$$

Dimension of $w^{[2]}$= (1, 4)

Vectorizing Input and All Variables



$$a^{[2]} = \sigma(z^{[2]})$$

$$z^{[2]} = w^{[2]}x + b^{[2]}$$

$$= w^{[2]}a^{[1]} + b^{[2]}$$

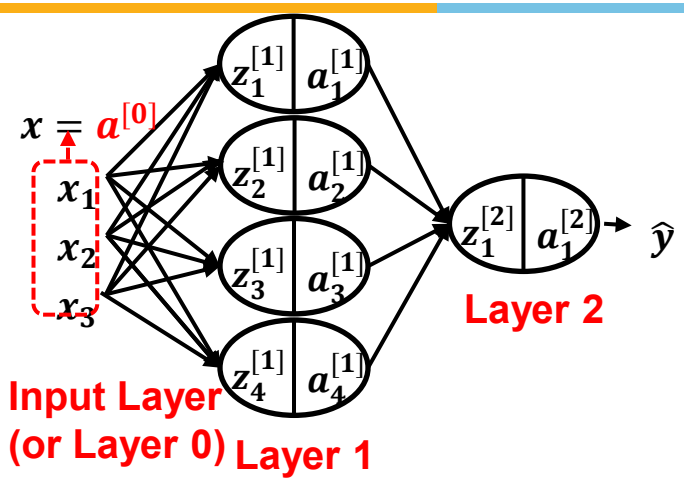
$$= \begin{bmatrix} w_{11}^{[2]} & w_{12}^{[2]} & w_{13}^{[2]} & w_{14}^{[2]} \end{bmatrix} \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ a_3^{[1]} \\ a_4^{[1]} \end{bmatrix} + \begin{bmatrix} b_1^{[2]} \end{bmatrix}$$

$$= \begin{bmatrix} w_{11}^{[2]} a_1^{[1]} + w_{12}^{[2]} a_2^{[1]} + w_{13}^{[2]} a_3^{[1]} + w_{14}^{[2]} a_4^{[1]} \end{bmatrix} + \begin{bmatrix} b_1^{[2]} \end{bmatrix}$$

$$a_1^{[2]} = \sigma(z_1^{[2]})$$

$$z_1^{[2]}$$

Vectorizing Input and All Variables



But, this assumes only 1 training example!

Forward propagation equations

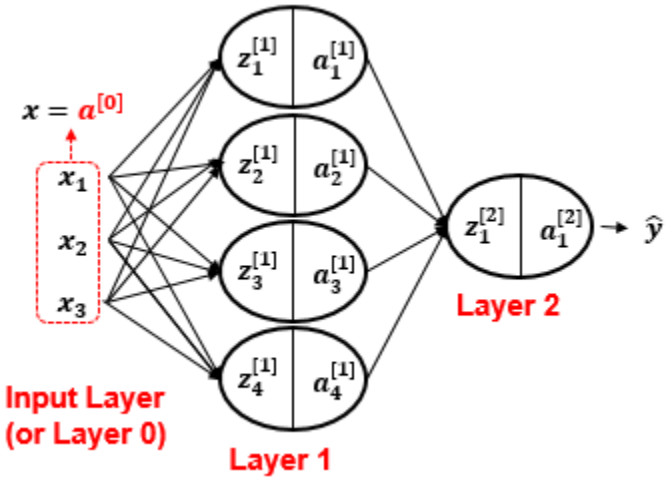
$$z^{[1]} = w^{[1]}x + b^{[1]}$$
$$a^{[1]} = \sigma(z^{[1]})$$
$$z^{[2]} = w^{[2]}a^{[1]} + b^{[2]}$$
$$a^{[2]} = \sigma(z^{[2]})$$

For l^{th} layer

$$z^{[l]} = w^{[l]}x + b^{[l]}$$
$$a^{[l]} = \sigma(z^{[l]})$$

Vectorizing Input and All Variables

Assuming m examples



$$for\ i = 1\ to\ m:$$

$$z^{[1](i)} = w^{[1]} a^{[0](i)} + b^{[1]}$$

$$a^{[1](i)} = \sigma(z^{[1](i)})$$

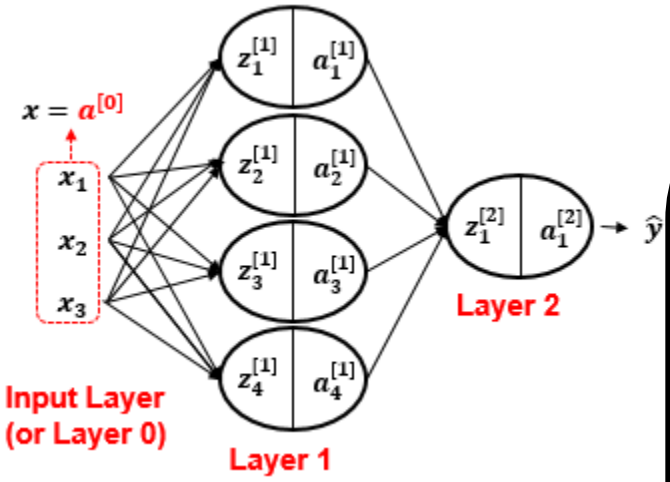
$$z^{[2](i)} = w^{[2]} a^{[1](i)} + b^{[2]}$$

$$a^{[2](i)} = \sigma(z^{[2](i)})$$

Refers to the i^{th} example in the training dataset

Vectorizing Input and All Variables

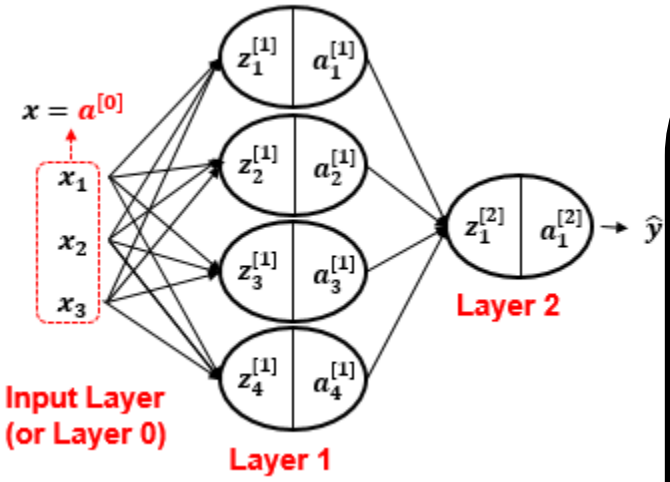
Assuming m examples



But, loops in general slow down programs; hence, it is better to further ***vectorize*** the implementation in order to avoid any loop, whenever possible

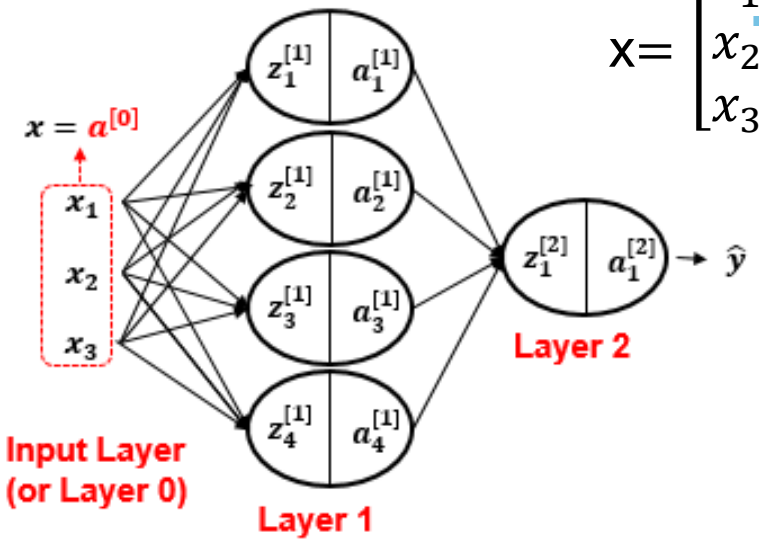
Vectorizing Input and All Variables

Assuming m examples



To this end, we can simply stack all $\boldsymbol{x}$ vectors (or $\boldsymbol{a^{[0]}}$ vectors), $\boldsymbol{z}$ vectors, and $\boldsymbol{a}$ vectors in different matrices of every layer!

Vectorizing Input and All Variables



$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

Assuming m examples

$$X = \begin{bmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \cdots & x^{(m)} \\ | & | & & | \end{bmatrix}$$

$$X = \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & & x_1^{(m)} \\ x_2^{(1)} & x_2^{(2)} & \cdots & x_2^{(m)} \\ x_3^{(1)} & x_3^{(2)} & & x_3^{(m)} \end{bmatrix}$$

This vector represents the *first* example in the training dataset

If we denote x as $a^{[0]}$

No. of features=n_x = 3

$$X = A^{[0]} = \begin{bmatrix} a_1^{[0](1)} & a_1^{[0](2)} & & a_1^{[0](m)} \\ a_2^{[0](1)} & a_2^{[0](2)} & \cdots & a_2^{[0](m)} \\ a_3^{[0](1)} & a_3^{[0](2)} & & a_3^{[0](m)} \end{bmatrix}$$

$A^{[0]}$ shape =(n_x, m)

Vectorizing Input and All Variables

Assuming m examples

Assuming m examples

$$Z^{[1]} = \begin{bmatrix} | & | & & | \\ z^{[1](1)} & z^{[2](2)} & \dots & z^{[1](m)} \\ | & | & & | \end{bmatrix}$$

No. of hidden units at layer 1 = $n_h = 4$

$Z^{[1]}$ shape = (n_h, m)

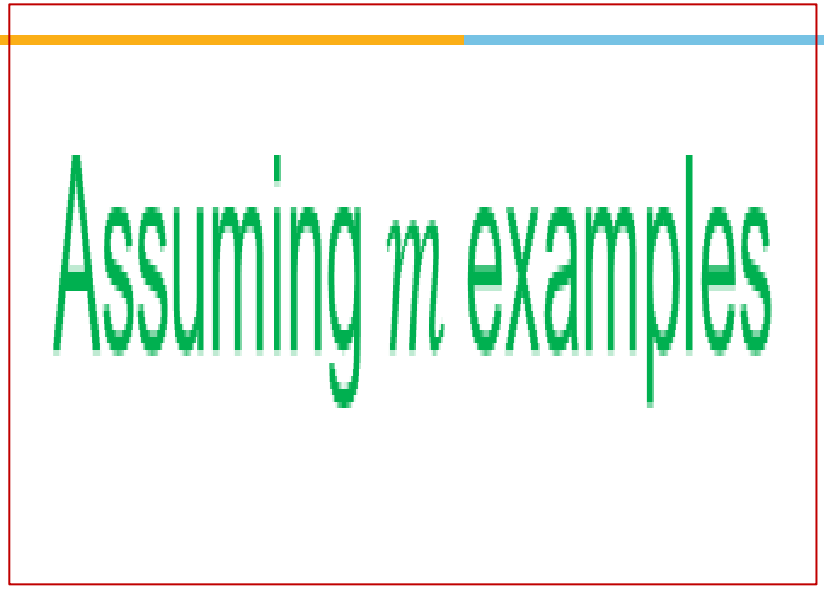
$$Z^{[1]} = w^{[1]} A^{[0]} + b^{[1]}$$

$$Z^{[1]} = matmul(w^{[1]}, A^{[0]}) + b^{[1]}$$

$$Z^{[1]} = \begin{bmatrix} z_1^{[1](1)} & z_1^{[1](2)} & & z_1^{[1](m)} \\ z_2^{[1](1)} & z_2^{[1](2)} & & z_2^{[1](m)} \\ z_3^{[1](1)} & z_3^{[1](2)} & \dots & z_3^{[1](m)} \\ z_4^{[1](1)} & z_4^{[1](2)} & & z_4^{[1](m)} \end{bmatrix}$$

Vectorizing Input and All Variables

Assuming m examples



$$A^{[1]} = \begin{bmatrix} | & | & & | \\ A^{[1](1)} & A^{[2](2)} & \dots & A^{[1](m)} \\ | & | & & | \end{bmatrix}$$

$A^{[1]}$ shape $=(n_h, m)$

$$A^{[1]} = \sigma(Z^{[1]})$$

$$A^{[1]} = \begin{bmatrix} a_1^{[1](1)} & a_1^{[1](2)} & & a_1^{[1](m)} \\ a_2^{[1](1)} & a_2^{[1](2)} & & a_2^{[1](m)} \\ a_3^{[1](1)} & a_3^{[1](2)} & \dots & a_3^{[1](m)} \\ a_4^{[1](1)} & a_4^{[1](2)} & & a_4^{[1](m)} \end{bmatrix}$$

Vectorizing Input and All Variables

Assuming m examples

Assuming m examples

$Z^{[2]}$ shape =(1,m)

$$Z^{[2]} = \begin{bmatrix} z_1^{[2](1)} & z_1^{[2](2)} & \dots & z_1^{[2](m)} \end{bmatrix}$$

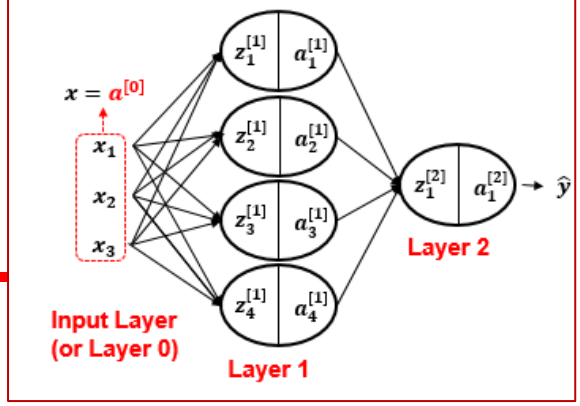
$$\boldsymbol{Z^{[2]} = w^{[2]} A^{[1]} + b^{[2]}}$$

$A^{[2]}$ shape =(1,m)

$$A^{[2]} = \begin{bmatrix} a_1^{[2](1)} & a_1^{[2](2)} & \dots & a_1^{[2](m)} \end{bmatrix}$$

$$\boldsymbol{A^{[2]} = \sigma(Z^{[2]})}$$

Forward propagation equations



$$for\ i = 1\ to\ n:$$

$$z^{[1](i)} = w^{[1]} a^{[0](i)} + b^{[1]}$$

$$a^{[1](i)} = \sigma(z^{[1](i)})$$

$$z^{[2](i)} = w^{[2]} a^{[1](i)} + b^{[2]}$$

$$a^{[2](i)} = \sigma(z^{[2](i)})$$

Before Vectorization

No Explicit Loop!

$$Z^{[1]} = w^{[1]} A^{[0]} + b^{[1]}$$

$$A^{[1]} = \sigma(Z^{[1]})$$

$$Z^{[2]} = w^{[2]} A^{[1]} + b^{[2]}$$

$$A^{[2]} = \sigma(Z^{[2]})$$

After Vectorization

Matrix dimensions

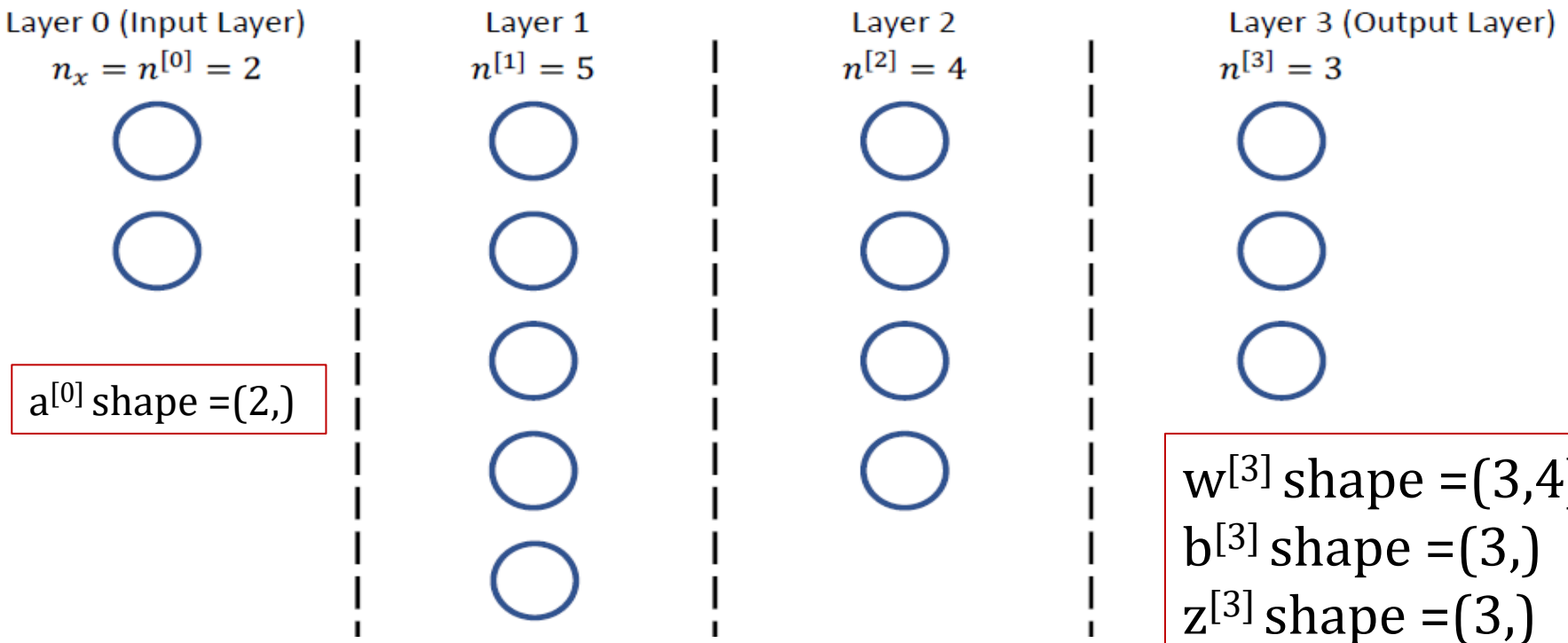
Notations

- L : Number of layers
- $n^{[l]}$: Number of units in layer l
- $w^{[l]}$: Weights for layer l
- $b^{[l]}$: Bias for layer l
- $z^{[l]}$: Hypothesis for layer l
- $g^{[l]}$: Activation Function used for layer l
- $a^{[l]}$: Activation for layer l

Matrix Dimensions

- $W^{[l]} : n^{[l]} \times n^{[l-1]}$
- $b^{[l]} : n^{[l]} \times 1$
- $z^{[l]} : n^{[l]} \times 1$
- $a^{[l]} : n^{[l]} \times 1$
- $dW^{[l]} : n^{[l]} \times n^{[l-1]}$
- $db^{[l]} : n^{[l]} \times 1$
- $dz^{[l]} : n^{[l]} \times 1$
- $da^{[l]} : n^{[l]} \times 1$

Dimensions for 1 training example:



$a^{[0]}$ shape =(2,)

$w^{[3]}$ shape =(3,4)
$b^{[3]}$ shape =(3,)
$z^{[3]}$ shape =(3,)
$\hat{y} = a^{[3]}$ shape =(3,)

$(n_h^{[l]}, n_x)$
$(n_h^{[l]},)$
$(n_h^{[l]},)$
$(n_h^{[l]},)$

$w^{[1]}$ shape =(5,2)
$b^{[1]}$ shape =(5,)
$z^{[1]}$ shape =(5,)
$a^{[1]}$ shape =(5,)

$w^{[2]}$ shape =(4,5)
$b^{[2]}$ shape =(4,)
$z^{[2]}$ shape =(4,)
$a^{[2]}$ shape =(4,)

Forward Step

- Given some [initial] values for the parameters, we can compute the forward pass, layer by layer.
- Forward pass is basically L matrix multiplications, each followed by an activation function.
- Matrix multiplication can be done efficiently with GPUs.
  - Therefore, forward pass is somewhat fast.
- Complexity of forward pass, linear of depth $O(L)$.

References

- Ref: Chapter 3 and 4 of T1
- http://neuralnetworksanddeeplearning.com/chap4.html

- https://cs229.stanford.edu/lectures-spring2022/main_notes.pdf

- http://neuralnetworksanddeeplearning.com/chap2.html

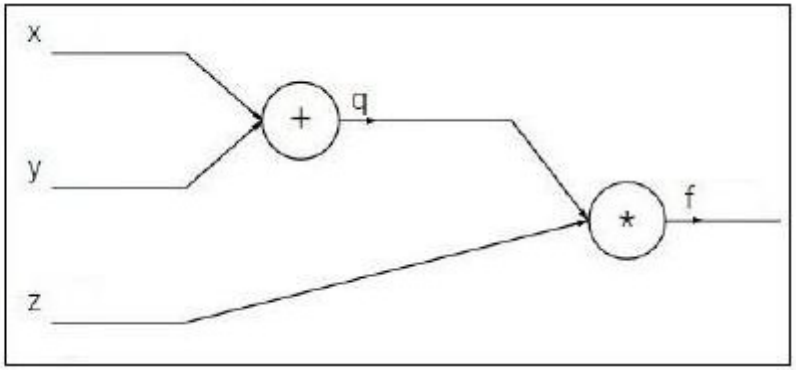
Practice problems

Example 1 Computational Graph for Back Propagation

$$f(x, y, z) = (x + y)z$$

Computational Graph for Back Propagation

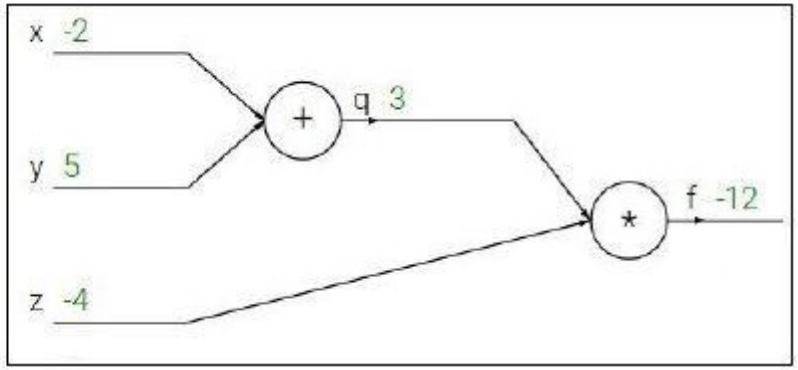
$$f(x, y, z) = (x + y)z$$



Computational Graph for Back Propagation

$$f(x, y, z) = (x + y)z$$

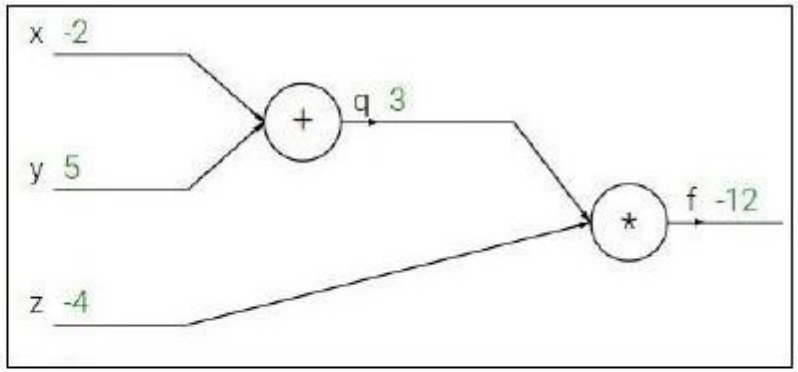
e.g. x = -2, y = 5, z = -4



Computational Graph for Back Propagation

$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4



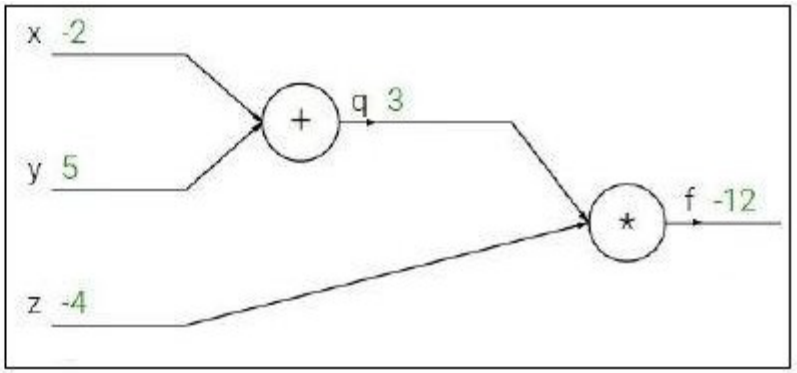
Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Computational Graph for Back Propagation

$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$



Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Computational Graph for Back Propagation

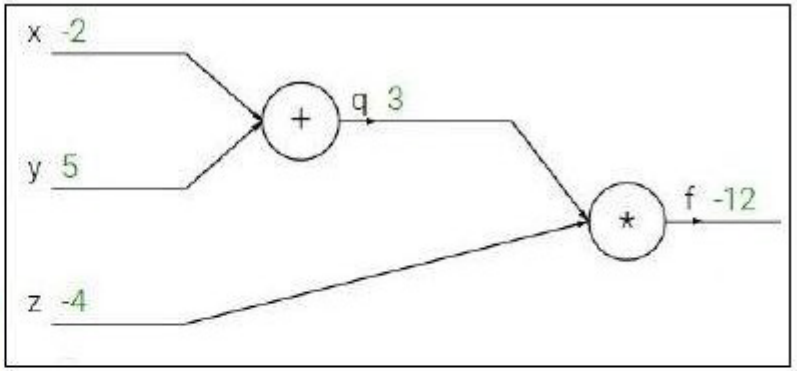
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Computational Graph for Back Propagation

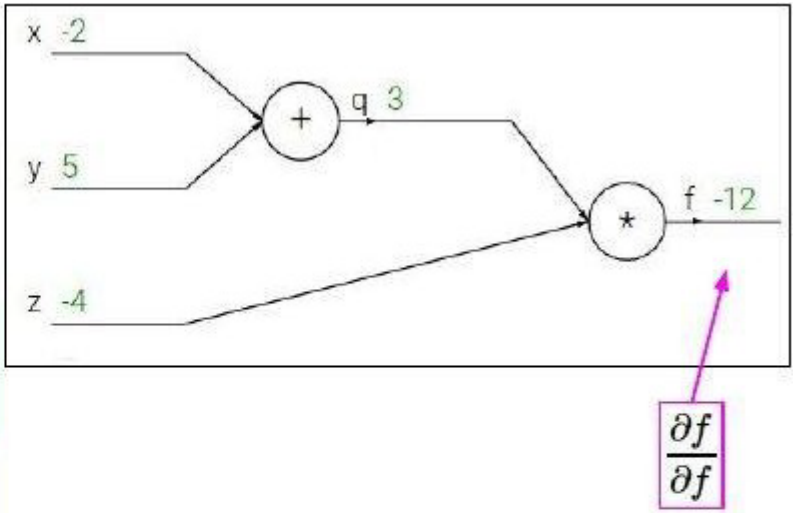
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Computational Graph for Back Propagation

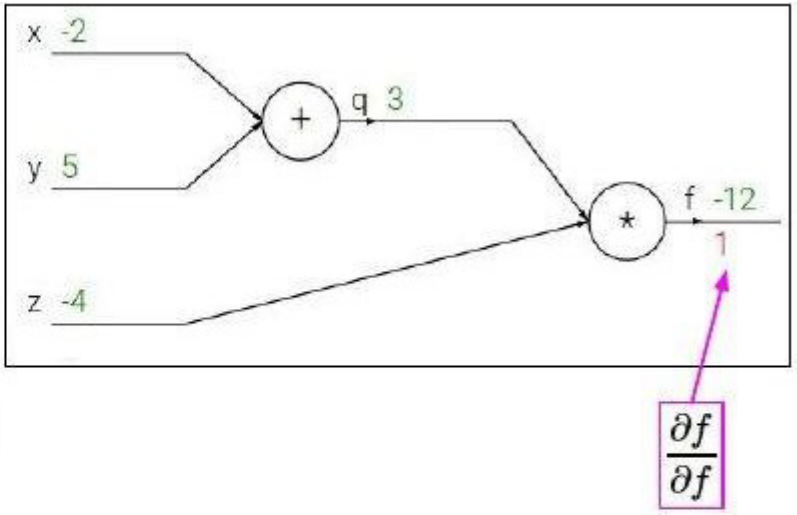
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Computational Graph for Back Propagation

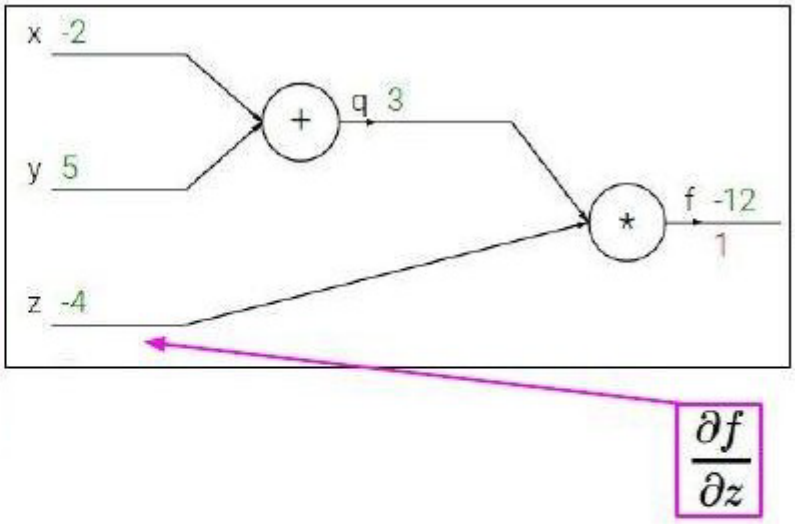
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Computational Graph for Back Propagation

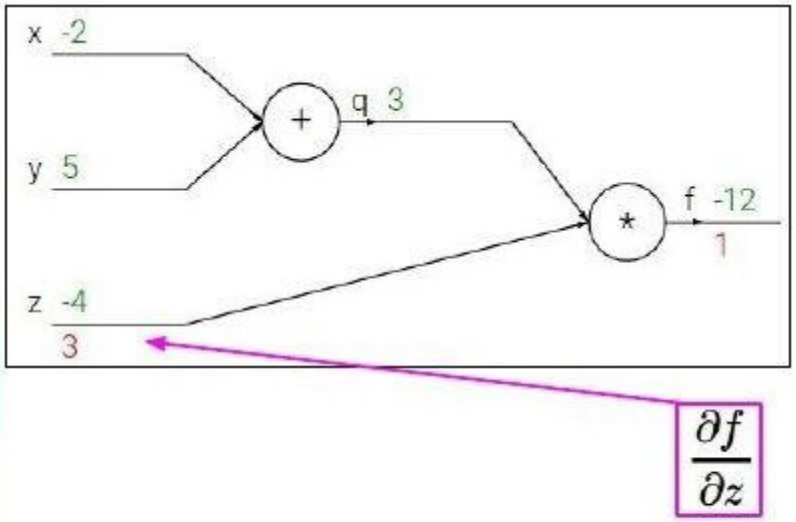
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Computational Graph for Back Propagation

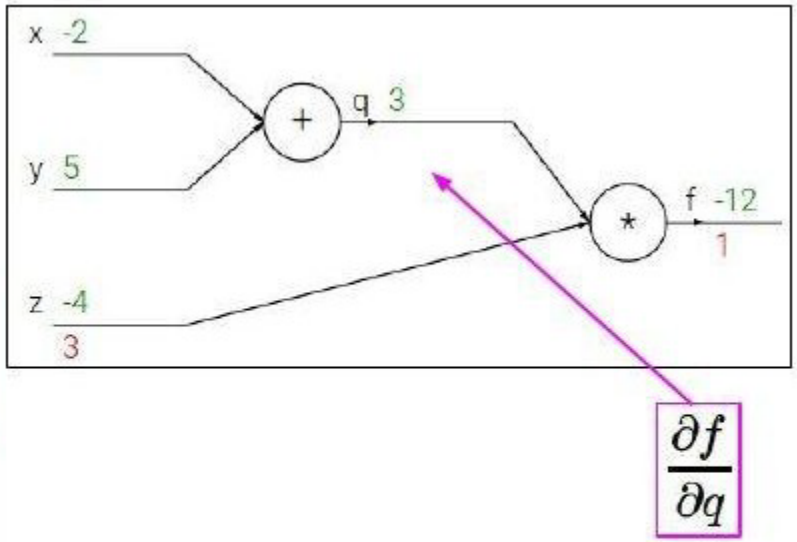
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Computational Graph for Back Propagation

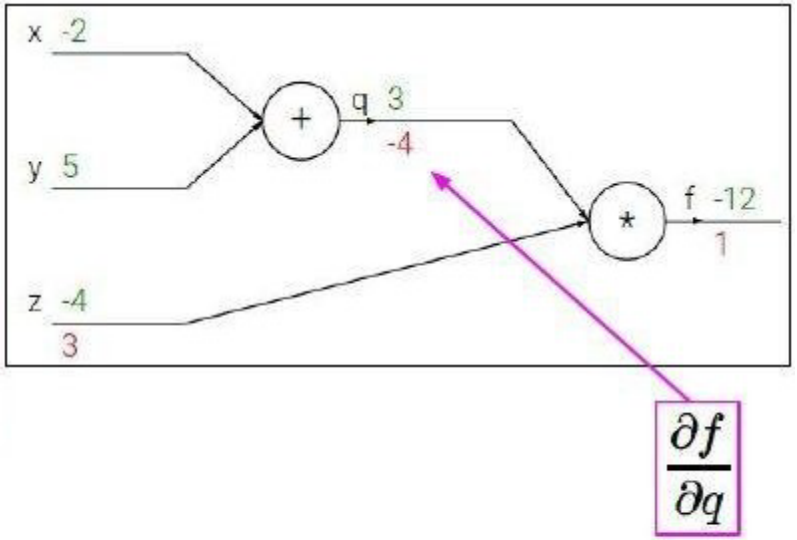
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Computational Graph for Back Propagation

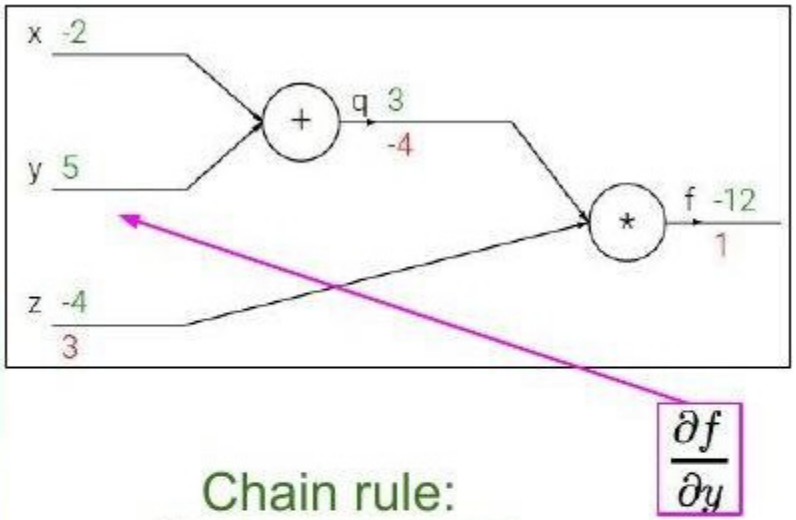
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Upstream gradient

Local gradient

Computational Graph for Back Propagation

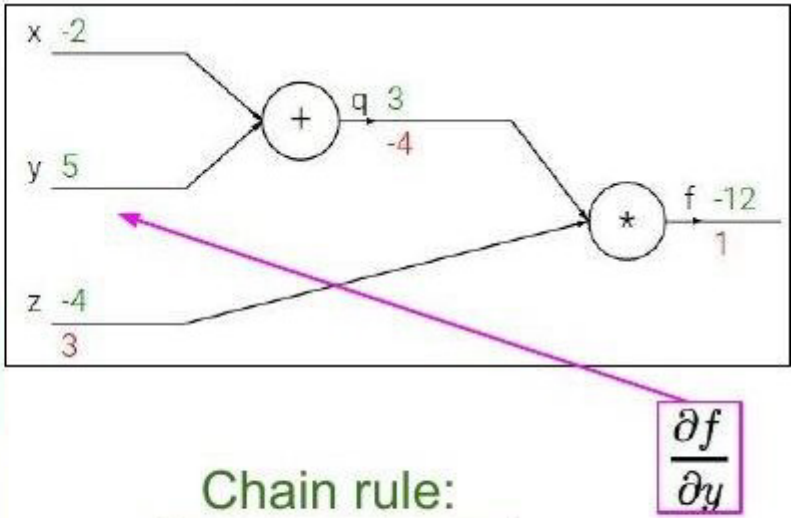
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Upstream gradient

Local gradient

Computational Graph for Back Propagation

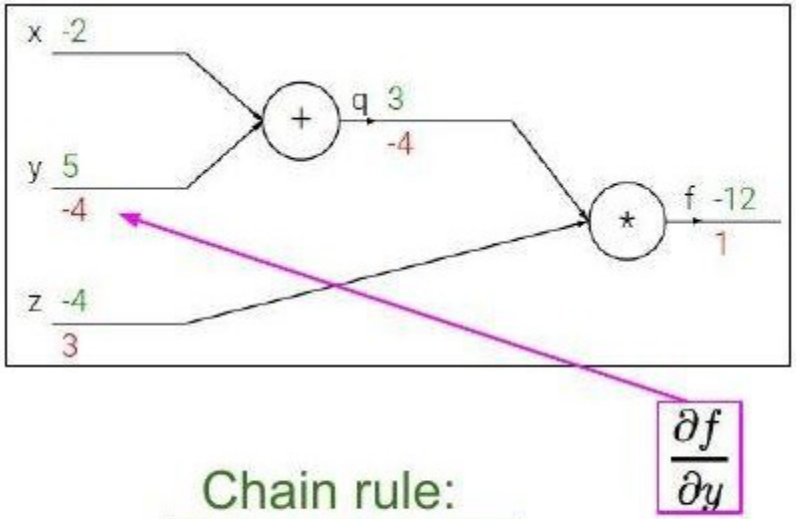
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Upstream gradient

Local gradient

Computational Graph for Back Propagation

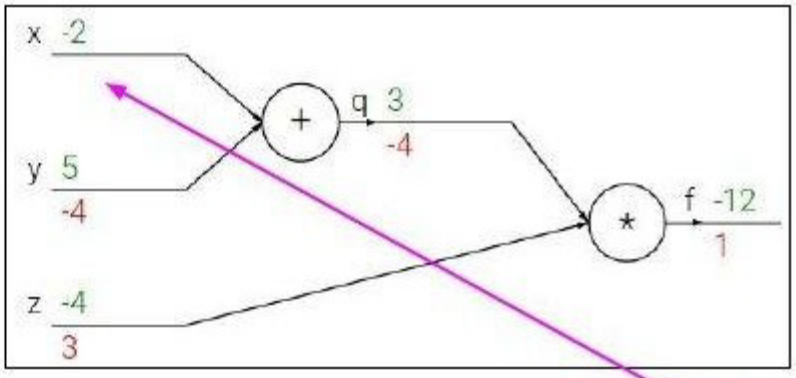
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x}$$

Upstream gradient

Local gradient

Computational Graph for Back Propagation

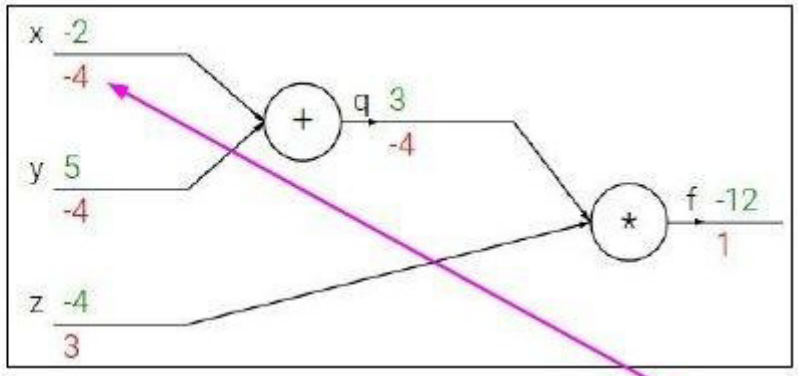
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

Upstream gradient

Local gradient

Example 2

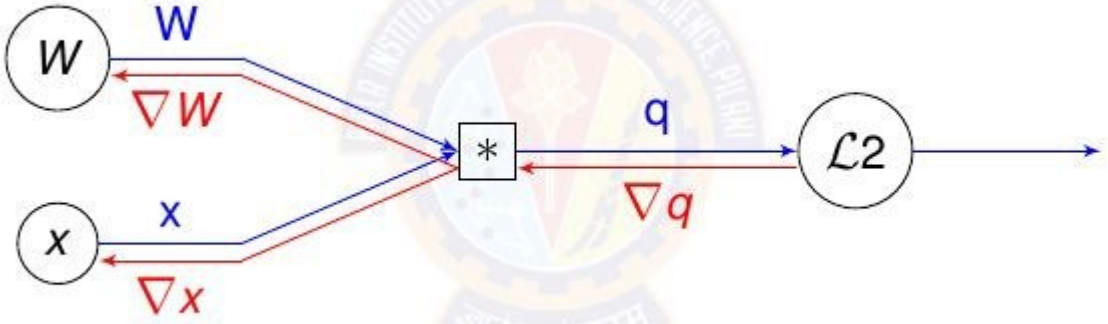
Draw the computational graph for the equation

$$f(x, w) = || W \cdot x ||^2 = \sum_{i=1}^{n} (W \cdot x)_i^2$$

Using the computation graph show the computation of the gradients also. Assume that

$$W = \begin{bmatrix} 0.1 & 0.5 \\ -0.3 & 0.8 \end{bmatrix}$$

$$x = \begin{bmatrix} 0.2 \\ 0.4 \end{bmatrix}$$


W
W
∇W
x
x
∇x
*
q
∇q
ℒ2

$$W = \begin{bmatrix} 0.1 & 0.5 \\ -0.3 & 0.8 \end{bmatrix}$$

$$x = \begin{bmatrix} 0.2 \\ 0.4 \end{bmatrix}$$

$$q = W \cdot x = \begin{bmatrix} 0.1 * 0.2 + 0.5 * 0.4 \\ -0.3 * 0.2 + 0.8 * 0.4 \end{bmatrix} = \begin{bmatrix} 0.22 \\ 0.26 \end{bmatrix}$$

$$L2 = \sum_{i=1} q_i^2 = 0.22^2 + 0.26^2 = 0.116$$

$$\nabla q = \nabla_q f = 2q$$
$$= 2 \begin{bmatrix} 0.22 \\ 0.26 \end{bmatrix} = \begin{bmatrix} 0.44 \\ 0.52 \end{bmatrix}$$

$$\nabla W = \nabla_W f = 2q \cdot x^\top$$
$$= 2 \begin{bmatrix} 0.22 \\ 0.26 \end{bmatrix} \begin{bmatrix} 0.2 & 0.4 \end{bmatrix} = \begin{bmatrix} 0.088 & 0.176 \\ 0.105 & 0.208 \end{bmatrix}$$

$$\nabla x = \nabla_x f = 2W^\top \cdot q$$
$$= 2 \begin{bmatrix} 0.1 & -0.3 \\ 0.5 & 0.8 \end{bmatrix} \begin{bmatrix} 0.22 \\ 0.26 \end{bmatrix} = \begin{bmatrix} -0.112 \\ 0.636 \end{bmatrix}$$

Thank You All !